

Mathias Bernat

CMPSC 132 – Final Project Report

## **Project Description**

My project is the terminal-based implementation of the Tic-Tac-Toe game. The program allows two players to take turns entering moves via user input, with the game displaying the board after each turn. The objective is to achieve three of the same symbols in a row, column, or diagonal before the opponent does. The game continues until either a player wins or the board is completely filled, resulting in a draw. Additionally, the program includes a replay feature that allows users to start a new game after one concludes.

## **Approach and Logic**

I immediately chose an object-oriented approach for my project implementation, with a central “Game” class representing the entire game. The class attributes include the game board (a two-dimensional list) and the current player, which alternates between 'X' and 'O'. The logic is divided into simple methods, such as “print\_board” for displaying the board, “move” for placing a player’s mark, “check\_win” for determining if a winning condition has been met, and “check\_draw” for identifying when the board is full. The play method acts as the main method of the program, coordinating the game loop, validating moves, and switching players. Input validation is handled using a loop combined with try/except handling to ensure that users provide valid integer inputs within the correct range and select unoccupied positions on the board.

## **Data Structures**

The primary data structure is a two-dimensional list to represent the Tic-Tac-Toe board. Each inner list corresponds to a row on the board, and each element within the row represents a cell that can hold a space character (empty), an 'X', or an 'O'. This structure allows for simple indexing when checking for winning conditions and updating positions on the board. Additionally, the use of a class encapsulates both the data (board and current player) and the related behaviors (methods), organizing the program into a single object. I feel that this object-oriented structure improves code readability by grouping related functionality together.

## **Challenges Faced**

The main challenge faced during development was organizing the overall structure of the program. Deciding how to divide responsibilities among methods and determining the flow of the game demanded more time than the actual coding itself. Another issue that arose was

related to replaying the game. An earlier version of my implementation risked causing a recursion error if players repeatedly chose to play again many times. This issue was resolved by adding another outermost loop to the play method that handled a "playing" variable to determine when the game is to be repeated.

### **Possible Improvements**

Potential improvements include improving the board display and adding input validation for the replay prompt. Currently, the program only continues if the user explicitly enters "Yes," and any other input causes the program to terminate. Adding a loop to force a valid "Yes" or "No" response would ensure that the result is intentional. A more significant improvement would be implementing a single-player mode with an AI-style opponent that reads the board and uses some more complex algorithms to select moves, maybe with an aspect of randomness.